SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning **COURSE CODE: MLA0405**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

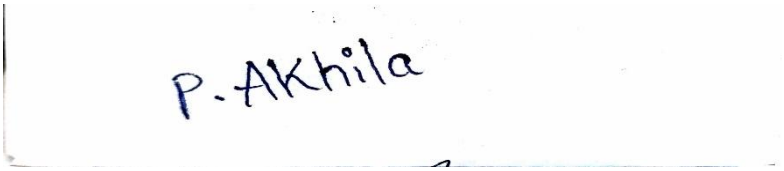
Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I **AKHILA.P (192425225) (**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

QUESTIONS: **Total Marks:25**

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.
b) Explain how this mistake affects parameter estimation.
c) Suggest the correct mathematical formulation and reasoning.
d) How would using log-likelihood fix computational issues?

Answer:

Problem Understanding

Maximum Likelihood Estimation (MLE) estimates model parameters by finding the values that maximize the likelihood of observing the given training data. In probability theory, independent sample probabilities must be multiplied, not added.

(a) Conceptual Error

The student incorrectly writes the likelihood function as:

$$L(\theta) = P(x_1) + P(x_2) + P(x_3) + \dots$$

This is mathematically incorrect because independent events require multiplication.
The correct likelihood is

$$L(\theta) = \prod_{i=1}^{n} P(x_i|\theta)$$

The product measures the joint probability of observing the entire dataset.

(b) Effect on Parameter Estimation

Because probabilities are added:

- Joint probability is not represented correctly.
- Estimated parameters no longer maximize the actual likelihood.
- Model converges to incorrect values.
- Prediction accuracy decreases.
- Even with large datasets, the estimated parameters remain biased.

This results in poor classification performance.

(c) Correct Mathematical Formulation

The correct likelihood function is

$$L(\theta) = \prod_{i=1}^{n} P(x_i|\theta)$$

Since multiplying many probabilities produces extremely small values, we convert it into log-likelihood.
The log-likelihood becomes

$$\log L(\theta) = \sum_{i=1}^{n} \log P(x_i|\theta)$$

This converts multiplication into addition while preserving the maximum point.

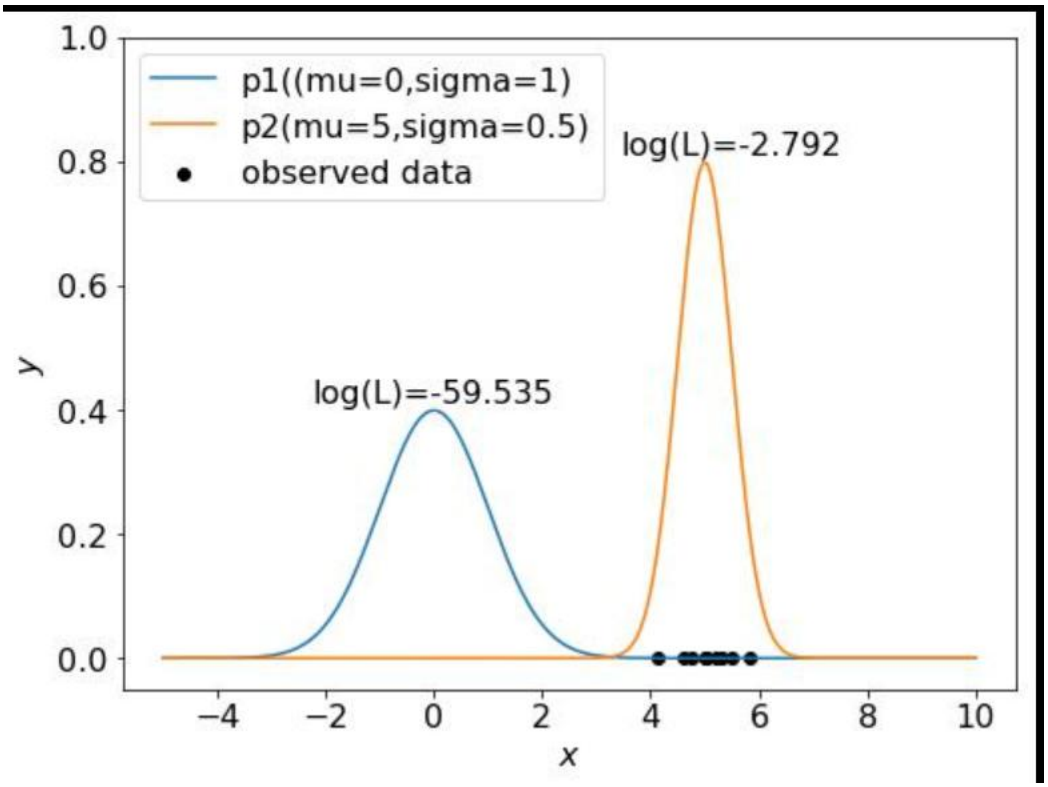
(d) Why Log-Likelihood Solves Computational Problems

Advantages include:

- Prevents numerical underflow.
- Easier differentiation.
- Faster optimization.
- Improves stability during Gradient Descent.
- More accurate parameter estimation.

Conclusion

MLE should always maximize the **product of probabilities** (or equivalently the **sum of log probabilities**). Using log-likelihood provides mathematically correct estimation and computational efficiency.



2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

b) Analyze how incorrect learning rate or weight updates affect convergence.

c) Identify dataset-related issues that may cause this problem.

d) Propose modifications to ensure convergence.

ANSWER:

Problem Understanding

The Perceptron is a supervised learning algorithm used for binary classification. It works correctly only when the dataset is **linearly separable**. If the model does not converge even after many iterations, it usually indicates implementation errors, incorrect parameter settings, or data-related issues.

(a) List Possible Implementation Errors in the Learning Algorithm

The following implementation mistakes can prevent convergence:

1. Incorrect Weight Update Rule

The correct Perceptron weight update is:

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

where:

- w= weight vector
- η= learning rate
- y= actual output
- $\hat{y}$= predicted output
- x= input feature vector

If this formula is implemented incorrectly, the model cannot learn.

2. Bias Not Updated

The bias term shifts the decision boundary. If it is not updated during training, classification accuracy decreases.

3. Wrong Activation Function

A Perceptron uses a **step (threshold) activation function**. Using sigmoid or another function without modifying the learning rule may prevent convergence.

4. Incorrect Label Encoding

The Perceptron typically expects labels as **+1 and −1** (or consistently 1 and 0). Incorrect encoding can confuse the learning algorithm.

5. Errors in Stopping Condition

Stopping training too early or setting too few epochs may prevent the algorithm from reaching convergence.

(b) Analyze How Incorrect Learning Rate or Weight Updates Affect Convergence

Incorrect Learning Rate

Very High Learning Rate

- Large weight updates.
- Overshoots the optimal decision boundary.
- Loss oscillates.
- Training becomes unstable.

Very Small Learning Rate

- Extremely slow learning.
- Requires many iterations.
- May stop before convergence.

Incorrect Weight Updates

If weights are updated in the wrong direction:

- Classification error increases.
- Decision boundary moves away from the correct solution.
- The model never converges.

Therefore, selecting an appropriate learning rate and correct update rule is essential.

(c) Identify Dataset-Related Issues That May Cause This Problem

Even with a correct algorithm, convergence may fail because of the dataset.

Possible Dataset Problems

- Dataset is **not linearly separable**.
- Presence of noisy or mislabeled samples.
- Duplicate or redundant records.
- Missing values.
- Highly imbalanced classes.
- Features have different scales.
- Irrelevant features increase complexity.

These issues reduce the Perceptron's ability to learn an effective decision boundary.

(d) Propose Modifications to Ensure Convergence

The following improvements help the Perceptron converge:

1. Normalize the Data

- Scale all features to a similar range.
- Prevents large feature values from dominating updates.

2. Choose an Appropriate Learning Rate

- Use a moderate learning rate (e.g., 0.01 or 0.1).
- Prevents oscillation while ensuring efficient learning.

3. Verify the Weight Update Rule

Ensure the implementation follows:

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

4. Update the Bias Correctly

The bias should be updated along with the weights after each misclassified sample.

5. Remove Noise and Outliers

Clean datasets improve convergence and classification accuracy.

6. Use More Powerful Models (if Needed)

If the data are not linearly separable, use:

- Multilayer Perceptron (MLP)
- Kernel-based Support Vector Machine (SVM)

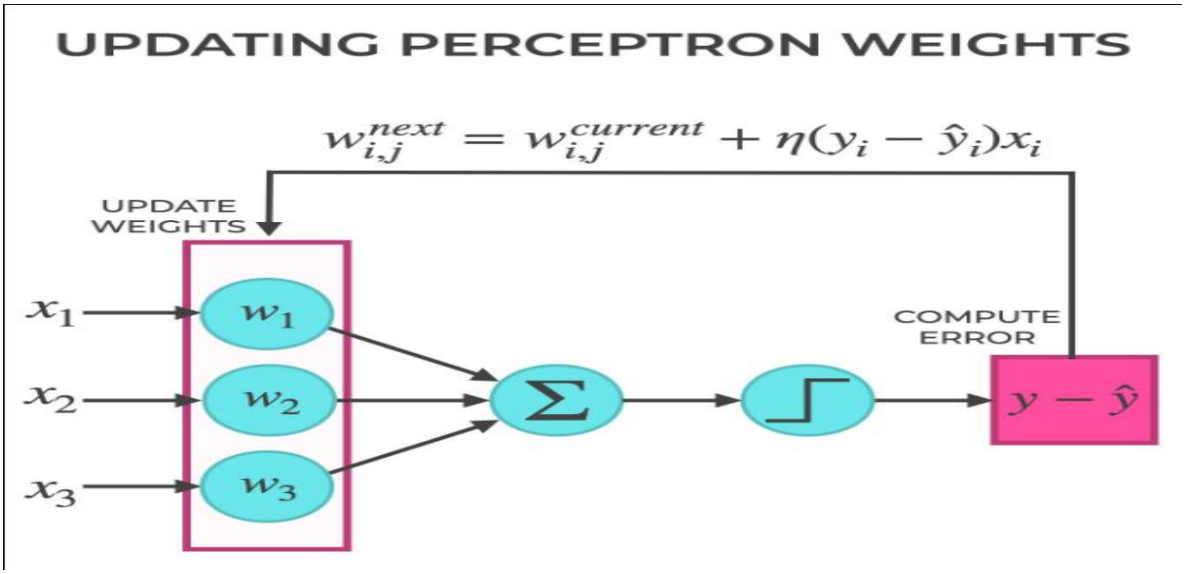
These models can learn nonlinear decision boundaries.

Advantages After Applying These Modifications

- Faster convergence.
- Stable training.
- Higher classification accuracy.
- Reduced training error.
- Better generalization on unseen data.

Conclusion

A Perceptron may fail to converge due to implementation mistakes, an inappropriate learning rate, incorrect weight updates, or poor-quality data. By using the correct update rule, selecting a suitable learning rate, preprocessing the dataset, and updating the bias correctly, the Perceptron can successfully converge on linearly separable datasets and produce accurate classification results.



3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.
b) Explain the role of activation functions in gradient flow.
c) Analyze how vanishing or exploding gradients affect training.
d) Suggest techniques to fix the issue (e.g., normalization, initialization).

ANSWER:

Problem Understanding

Backpropagation is the learning algorithm used to train Multilayer Perceptrons (MLPs). It computes gradients of the loss function with respect to each weight and updates them to reduce the training error. If the loss increases instead of decreasing, there is likely an error in gradient computation, learning rate selection, or parameter initialization.

(a) Identify Possible Mathematical or Coding Errors in Gradient Computation

Possible errors include:

- Incorrect derivative of the activation function.
- Wrong sign in gradient descent update.
- Incorrect chain rule implementation.
- Updating weights before computing all gradients.
- Wrong indexing of neurons or layers.
- Learning rate set too high.
- Incorrect loss function implementation.

Correct weight update:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

where:

- w= weight
- η= learning rate
- L= loss function

(b) Explain the Role of Activation Functions in Gradient Flow

Activation functions introduce non-linearity into the network.

Common activation functions:

- Sigmoid
- Tanh
- ReLU

Their role:

- Determine how gradients flow backward.
- Affect learning speed.
- Influence convergence.

For example:

- Sigmoid may produce very small gradients.
- ReLU avoids vanishing gradients for positive inputs.
- Tanh provides zero-centered outputs but may still suffer from vanishing gradients.

(c) Analyze How Vanishing or Exploding Gradients Affect Training

Vanishing Gradient

- Gradients become extremely small.
- Early layers learn very slowly.
- Training almost stops.
- Loss decreases very slowly.

Exploding Gradient

- Gradients become excessively large.
- Weight updates become unstable.
- Loss increases rapidly.
- Model may diverge.

(d) Techniques to Fix the Issue

1. Xavier or He Initialization

- Initializes weights appropriately.
- Prevents unstable gradients.

2. Batch Normalization

- Normalizes intermediate outputs.
- Stabilizes learning.

3. Gradient Clipping

- Limits excessively large gradients.

4. ReLU Activation

- Reduces vanishing gradients.

5. Proper Learning Rate

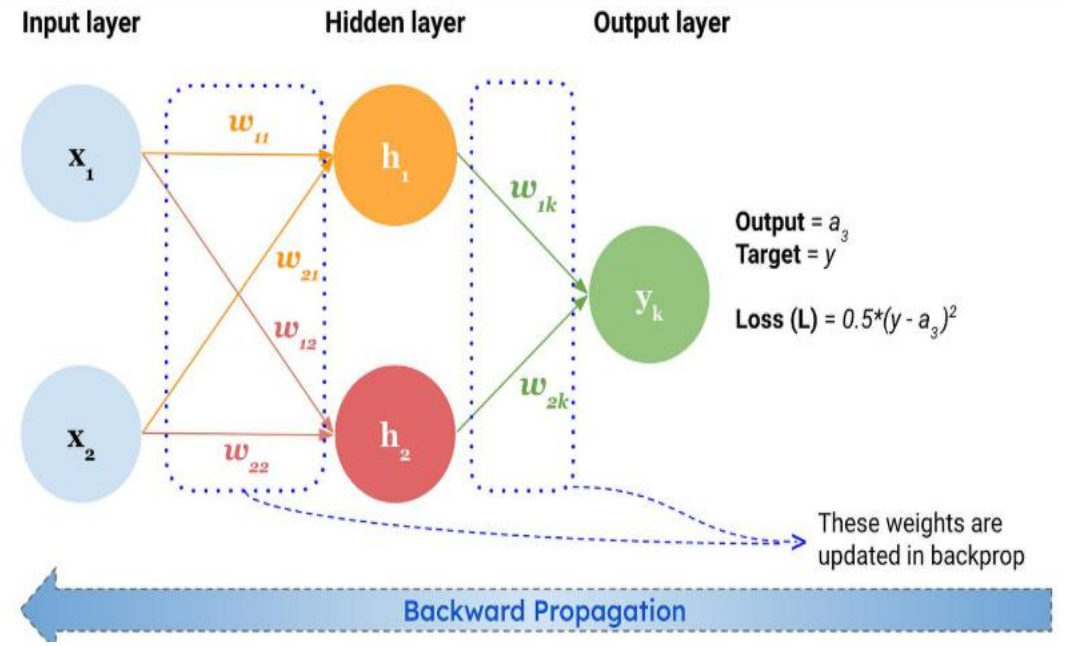
- Prevents unstable updates.

6. Adam Optimizer

- Adapts learning rates automatically.
- Faster convergence than standard SGD.

Conclusion

Increasing loss during backpropagation is usually caused by incorrect gradients, poor initialization, or unsuitable activation functions. Using proper initialization, normalization, gradient clipping, and adaptive optimizers ensures stable training and better model accuracy.



4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.

b) Analyze the cause of fluctuations in SGD.

c) Compare the error behavior of Mini-Batch Gradient Descent.

d) Recommend the best approach and justify with error trade-offs.

ANSWER:

Problem Understanding

Gradient Descent (GD) and Stochastic Gradient Descent (SGD) are optimization algorithms used to minimize the loss function. Batch Gradient Descent computes gradients using the entire dataset, while SGD updates weights after every training example. Mini-Batch Gradient Descent combines the advantages of both methods.

(a) Explain Why Batch Gradient Descent is Slow in Large Datasets

Batch Gradient Descent processes the complete dataset before updating weights.

This leads to:

- High computational cost.
- Large memory usage.
- Slow parameter updates.
- Longer training time.

Although it provides smooth convergence, it is inefficient for very large datasets.

(b) Analyze the Cause of Fluctuations in SGD

SGD updates weights after each training sample.

Advantages:

- Faster updates.
- Escapes shallow local minima.

Disadvantages:

- Noisy gradient estimates.
- Loss fluctuates during training.
- Requires more epochs to stabilize.

These fluctuations occur because each update is based on only one sample instead of the entire dataset.

(c) Compare the Error Behavior of Mini-Batch Gradient Descent

Mini-Batch Gradient Descent uses a small subset of samples (e.g., 32 or 64).

Comparison:

Method	Error Behavior
Batch GD	Smooth but slow convergence
SGD	Fast but highly noisy
Mini-Batch GD	Stable and faster convergence

Mini-Batch GD provides a balance between speed and stability.

(d) Recommend the Best Approach and Justify with Error Trade-Offs

For most deep learning applications:

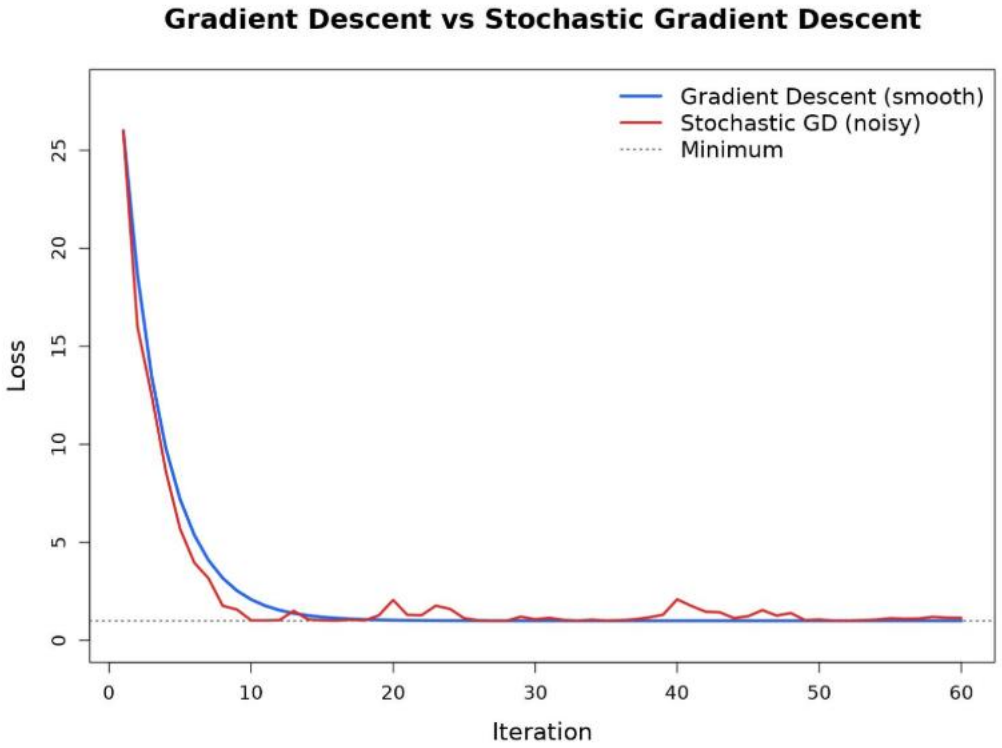
Mini-Batch Gradient Descent is recommended because it:

- Reduces computational cost.
- Improves convergence speed.
- Produces more stable updates than SGD.
- Requires less memory than Batch GD.
- Generalizes well on unseen data.

Common batch sizes: **32, 64, or 128**.

Conclusion

Batch Gradient Descent is accurate but slow, SGD is fast but noisy, while Mini-Batch Gradient Descent combines the strengths of both, making it the preferred optimization algorithm for deep learning.



5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

b) Identify signs of overfitting in this scenario.

c) Analyze why distance-based learning fails in high dimensions.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

ANSWER:

Problem Understanding

The **Curse of Dimensionality** occurs when the number of features becomes very large compared to the number of training samples. High-dimensional data often causes overfitting, poor generalization, and increased computational complexity.

(a) Explain How High Dimensionality Increases Model Error

When the number of features increases:

- Data become sparse.
- More training samples are required.
- Noise increases.
- The model memorizes training data.
- Test accuracy decreases.

This results in poor generalization.

(b) Identify Signs of Overfitting

Common signs include:

- Very high training accuracy.
- Low testing accuracy.
- Large gap between training and testing performance.
- High model complexity.
- Poor prediction on unseen data.

These indicate that the model has learned noise instead of meaningful patterns.

(c) Analyze Why Distance-Based Learning Fails in High Dimensions

Algorithms such as:

- K-Nearest Neighbors (KNN)
- K-Means Clustering

depend on distance calculations.

In high dimensions:

- Distances between points become very similar.
- Nearest neighbors become difficult to distinguish.
- Similarity measures lose effectiveness.
- Classification accuracy decreases.

This phenomenon reduces the effectiveness of distance-based algorithms.

(d) Suggest Dimensionality Reduction or Regularization Techniques

1. Principal Component Analysis (PCA)

- Reduces feature dimensions.
- Preserves most important information.
- Improves computational efficiency.

2. Feature Selection

- Removes irrelevant features.
- Reduces noise.
- Simplifies the model.

3. Regularization (L1/L2)

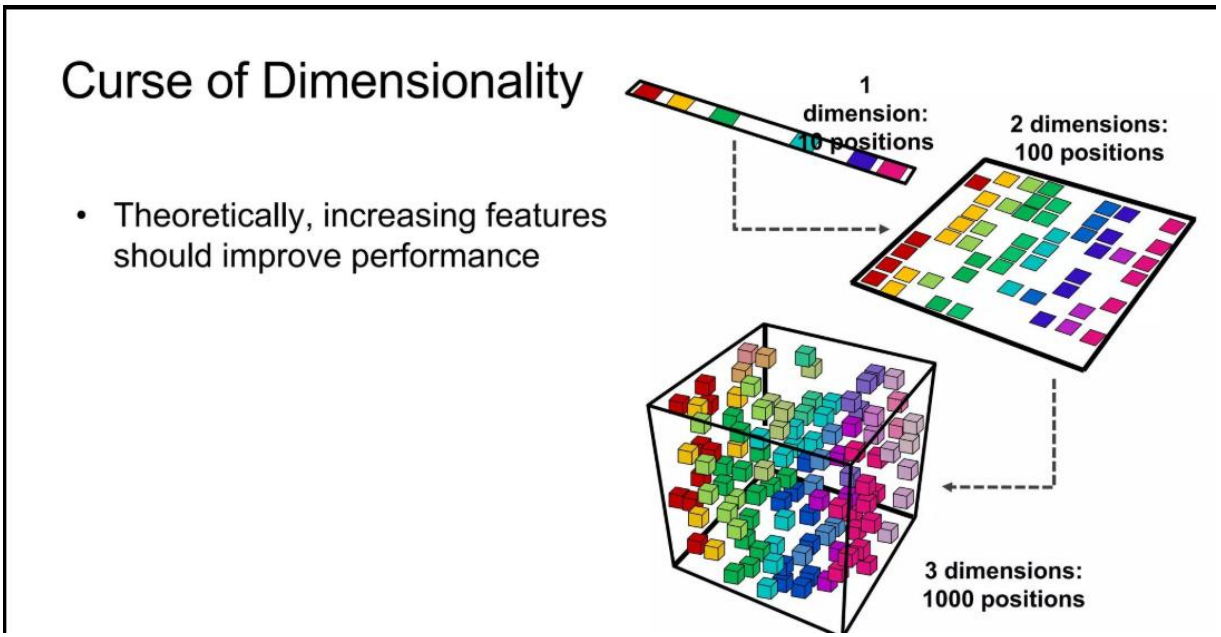
- Penalizes large weights.
- Reduces overfitting.
- Improves generalization.

4. Dropout

- Randomly disables neurons during training.
- Prevents co-adaptation.
- Enhances model robustness.

Conclusion

The Curse of Dimensionality increases model error, slows training, and causes overfitting. Applying PCA, feature selection, regularization, and dropout reduces complexity, improves generalization, and enhances overall model performance.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand